Javi User Manual

A Vi-Like Editor Written in Java

May 2026

Contents

1 Introduction

Javi is a vi-like text editor written in Java using AWT for its graphical interface. It provides the familiar modal editing experience of vi with modern extensions: an integrated VT100 terminal emulator, persistent undo history, a directory editor, Language Server Protocol (LSP) support, Git integration, and AI-assisted coding via GitHub Copilot.

1.1 Starting Javi

Launch Javi from the command line:

```
java -jar javi-all.jar [file ...]
```

Open one or more files by name. If no files are given, Javi starts with an empty buffer.

1.2 Modal Editing

Like vi, Javi is a *modal* editor with two primary modes:

- **Command mode** (default) — keystrokes are interpreted as commands (movement, deletion, yanking, etc.).
- **Insert mode** — keystrokes insert text into the buffer. Enter insert mode with `i`, `a`, `o`, etc. Return to command mode with `Escape`.

All vi conventions apply: `h` `j` `k` `l` for movement, `d` `d` to delete a line, `y` `y` to yank, `p` to paste, and so on.

Screenshot: *Javi main editor window showing a Java source file in command mode*

1.3 Help System

Javi has a built-in help system accessible from command mode:

Command	Description
`:help`	Show help index
`:help movement`	Cursor movement commands
`:help editing`	Text editing commands
`:help search`	Search and replace
`:help files`	File and buffer management
`:help ex`	Ex (colon) commands
`:help visual`	Visual selection mode
`:help undo`	Undo and redo
`:help window`	Window and scrolling
`:help shell`	Shell / terminal commands
`:help diredit`	Directory editor
`:help filelist`	File list buffer
`:help directory`	Directory list buffer
`:help keybindings`	Key binding architecture
`:help folding`	Folding commands and detection
`:help git`	Git integration commands

Help content is displayed in a read-only buffer navigable with normal vi movement keys.

1.4 Context-Sensitive Help Panel

Javi includes a context-sensitive help side panel that dynamically shows the key bindings available in the current context. Unlike `:help` topics which display static documentation, the context help panel queries the live keymap system and updates as you switch buffers or modes.

Command	Description

`Shift-F1`	Toggle context-sensitive help side panel
`:contexthelp`	Toggle context-sensitive help side panel
`:helpscrolldown`	Scroll help panel down
`:helpscrollup`	Scroll help panel up

The help panel appears as a fixed-width side panel on the right edge of the editor window. It does not take keyboard focus — you can continue editing while the panel is visible.

The panel content adapts to your current context:

- **Normal mode** — shows all bound keys from the active keymap chain, including any overlay bindings (e.g., DirEdit or file list bindings).
- **Insert mode** — shows insert-mode key bindings.
- **Sub-mode** (e.g., after pressing `d` or `z`) — shows the available completions for the pending operator.

> **Tip:** Leave the help panel open while learning Javi. It updates automatically as you switch between buffers with different keymap overlays (shell, directory editor, file list, etc.).

2 Differences from Vi

Javi is modeled on POSIX vi, not vim. If you know basic vi, you already know how to use Javi. This section describes where Javi differs from or extends the original vi specification.

2.1 Persistent Undo

Vi provides a single level of undo. Javi supports **unlimited, persistent undo**. Your entire undo history is saved in `.dmp2` files alongside each edited file. You can undo changes even after closing and reopening a file.

Command	Description
`u`	Undo last change
`Ctrl-R`	Redo last undone change
`Ctrl-Z`	Undo (alternate binding)
`Ctrl-Y`	Redo (alternate binding)
`U`	Undo all changes on current line

2.2 Regular Expressions

Vi uses Basic Regular Expressions (BRE) with escaped grouping (`\(` and `\)`). Javi uses **Java regular expressions** (similar to PCRE), which are more powerful:

- `.` matches any character, `\w` matches word characters, `\s` matches whitespace
- `+` means one-or-more (no escaping needed)
- `[abc]` character classes work as expected
- `^` and `$` anchor to line boundaries

2.3 Function Keys

Vi has no function key bindings. Javi maps function keys to frequently used features:

Command	Description
`F1`	Next position in position list
`F2`	File list (open buffers)
`F3`	Directory list
`F4`	Font list
`F5`	Position list
`F6`	Plugin list
`F7`	Make (build)

`F8`	Terminal / shell
`F11`	Toggle fullscreen

2.4 Word Movement

Javi supports the full set of vi word-motion commands:

Command	Description
`w` / `W`	Forward to start of word / WORD
`b` / `B`	Backward to start of word / WORD
`e` / `E`	Forward to end of word / WORD

A *word* is a sequence of letters, digits, and underscores. A *WORD* is any sequence of non-whitespace characters.

2.5 Character Search on Line

Command	Description
`f` *char*	Find *char* forward on current line
`F` *char*	Find *char* backward on current line
`t` *char*	To *char* forward (cursor before *char*)
`T` *char*	To *char* backward (cursor after *char*)
`;`	Repeat last f/F/t/T
`,`	Repeat last f/F/t/T in opposite direction

2.6 Visual Selection

Vi does not have visual mode. Javi adds visual selection similar to vim:

Command	Description
`v`	Enter character-wise visual mode
`V`	Enter line-wise visual mode

In visual mode, use movement keys to extend the selection, then apply an operator:

Command	Description
`d`	Delete selection
`y`	Yank (copy) selection
`c`	Change selection (delete + insert)
`>` / `<`	Shift selection right / left
`~`	Toggle case
`Escape`	Exit visual mode

2.7 Directory Editor (DirEdit)

Vi has no built-in directory browsing. Javi provides a netrw-like directory editor:

```
:e .          Open current directory
:diredit /path   Open specific directory
```

Command	Description
`Enter`	Open file or enter directory
`-`	Go to parent directory
`.`	Toggle hidden (dot) files
`s`	Cycle sort mode (name / size / date / type)
`R`	Refresh listing

`D`	Delete file under cursor (with confirmation)
`S`	Toggle directory in/out of search path
`q`	Quit directory browser

Screenshot: *Directory editor showing a file listing with sort and hidden-file indicators*

File operations via colon commands:

Command	Description
`:diredit_rename _name_`	Rename file under cursor
`:diredit_mkdir _name_`	Create new subdirectory
`:diredit_newfile _name_`	Create new empty file
`:diredit_copy _dest_`	Copy file under cursor

2.8 Integrated Terminal

Vi requires you to leave the editor to run shell commands (`:!cmd` executes and returns). Javi includes a full VT100 terminal emulator inside the editor. See Section 5 for details.

2.9 Tags and ID Lookup

Javi supports ctags-based navigation for jumping to definitions, and mkid (GNU ID Utils) for finding references:

Command	Description
`Ctrl-]`	Jump to definition (LSP first, then ctags)
`Ctrl-T`	Pop tag stack (return to previous location)
`:ta _tag_`	Jump to named tag (LSP first, then ctags)

When the LSP plugin is loaded, `Ctrl-]` and `:ta` try LSP go-to-definition first. If no LSP server is running or it returns no result, the lookup falls through to ctags. When looking up via ctags, Javi also automatically searches the `mkid` ID database (built with GNU ID Utils) to find references. Both ctag definitions and `lid` cross-references are combined in the tag results buffer.

Not yet implemented: The commands `:tagsauto`, `:tagfiles`, `:tagadd`, and `:gid` are planned but do not exist in the current codebase. Tag files are loaded from the default `tags` file in the project directory.

2.10 Key Binding Customization

Vi has limited `:map` support. Javi provides a layered keymap architecture:

Command	Description
`:mapkey _group_ _key_ _command_`	Bind a key
`:unmapkey _group_ _key_`	Unbind a key
`:keymap`	Show active keymap chain
`:map`	Show all key bindings
`:savemapkeys`	Save user bindings to disk
`:loadmapkeys`	Load user bindings from disk

group is either `move` (movement keys) or `edit` (editing/command keys). *key* can be a single character, `C-x` (Ctrl), `S-x` (Shift), or names like `F1`–`F12`, `Up`, `Down`, `Home`, `End`, `PgUp`, `PgDn`.

Bindings are saved to `~/.javi/keybindings`. To auto-load on startup, add `loadmapkeys` to your `.javini` file.

2.11 File List (F2)

Pressing `F2` opens the file list, showing all files in the editor. The file list uses an overlay keymap where `Enter` opens the file at the cursor instead of moving down a line. All other normal-mode keys work as usual.

Command	Description
`Enter` / `F1`	Open file at cursor
`Ctrl-F1`	Open and show position list
`Shift-F1`	Open in split view
`F2`	Return to previous buffer

2.12 Settings

Command	Description
`:set _option_=_value_`	Set an editor option
`:tabstop _n_`	Set tab display width
`:lines _n_`	Set default window height
`:setwidth _n_`	Set default window width

3 Basic Commands Reference

This section provides a condensed reference of the most commonly used commands. For the full reference, use `:help _topic_` within Javi.

3.1 Movement

Command	Description
`h` `j` `k` `l`	Left, down, up, right
`0` `^` `$`	Start of line, first non-blank, end of line
`w` `b` `e`	Word forward, word backward, end of word
n `G`	Go to line *n* (e.g., `1G` = first line, `G` alone = last line)
`Shift-Home` / `Ctrl-Home`	Go to first line
`H` `M` `L`	Top, middle, bottom of screen
`Ctrl-F` / `Ctrl-B`	Page forward / backward
`Ctrl-D` / `Ctrl-U`	Half-page down / up
`%`	Jump to matching bracket

3.2 Editing

Command	Description
`i` / `a`	Insert before / append after cursor
`I` / `A`	Insert at line start / append at line end
`o` / `O`	Open line below / above
`x`	Delete character under cursor
`dd`	Delete entire line
`d` *motion*	Delete with motion (e.g., `dw`, `d$`)
`cc`	Change entire line
`c` *motion*	Change with motion
`r` *char*	Replace single character
`R`	Enter replace (overwrite) mode
`yy`	Yank (copy) entire line

`p` / `P`	Paste after / before cursor
`J`	Join current line with next
`.`	Repeat last change
`>>` / `<<`	Shift current line right / left

3.3 Search and Replace

Command	Description
/pattern	Search forward
?*pattern*	Search backward
`n` / `N`	Repeat search same / opposite direction
`:s/old/new/`	Substitute first occurrence on current line
`:s/old/new/g`	Substitute all on current line
`:%s/old/new/g`	Substitute all in entire file

3.4 File Operations

Command	Description
`:e _file_`	Edit file
`:e!`	Reload current file (discard changes)
`:w`	Write (save) file
`:w _file_`	Write to named file
`:wq`	Write and quit
`:q`	Quit (fails if unsaved changes)
`:q!`	Quit without saving
`:r _file_`	Read file contents into buffer
`Ctrl-G`	Show file status information
`Ctrl-^`	Switch to next file in file list

3.5 Ex Commands

Javi implements a subset of POSIX ex commands. For a complete reference to traditional ex/vi commands, see the POSIX ex specification.

Command	Description
`:_n_`	Go to line *n*
`:_n_,_m_ d`	Delete lines *n* through *m*
`:_n_,_m_ y`	Yank lines *n* through *m*
`:_n_,_m_ m _k_`	Move lines to after line *k*
`:g/_pattern_/d`	Delete all lines matching *pattern*
`:v/_pattern_/d`	Delete all lines NOT matching *pattern*
`:!_cmd_`	Run external shell command
`:mk`	Run make (build)

Differences from POSIX ex:

- Regular expressions use Java syntax (see Section 2) instead of POSIX BRE.
- The `global` command (`g/pattern/cmd`) supports only `d` (delete); arbitrary ex commands after the pattern are not supported.
- The `open` and `visual` mode transitions from ex are not implemented — Javi always starts in visual (normal) mode.
- Address forms `.` (current line) and `$` (last line) are supported; `'a` (mark) addresses and relative offsets (`+n`, `-n`) are not.

- The `set` command uses = syntax (`:set option=value`) rather than the POSIX toggle form.

4 Folding

Folding lets you collapse regions of text into a single summary line, reducing visual clutter when working with large files. Collapsed regions hide their contents from view but remain in the buffer — unfolding restores the full text. Fold state is preserved across sessions via `.foldstate` files.

4.1 Fold Detection Methods

Before using fold commands, you must detect folds using one of the three detection methods. Each creates a set of fold regions based on different heuristics.

4.1.1 Syntax-Based Folding (`:fold`)

The `:fold` command detects folds by matching braces and brackets. Every matched `{…}` or `[…]` pair that spans multiple lines becomes a foldable region. This works well for JSON, Java, C, and other brace-delimited languages.

```
:fold
```

4.1.2 Indent-Based Folding (`:foldindent`)

The `:foldindent` command creates folds based on indentation level, similar to vim's `foldmethod=indent`. Lines at deeper indentation are folded under lines at shallower indentation. Blank lines inherit the minimum indent level of their surrounding non-blank lines so they do not break folds.

```
:foldindent           " use default tab size (3)
:foldindent 4         " use 4-space indent levels
```

4.1.3 Marker-Based Folding (`:foldmarker`)

The `:foldmarker` command detects explicit fold markers in the text. Place `\{\{\{` to start a fold region and `\}\}\}` to end it. Markers can include an optional level number (e.g., `\{\{\{1`, `\}\}\}1`) for nested folds. This gives you precise control over which regions are foldable.

```
:foldmarker
```

Example in source code:

```
// Section A {{{1
... code ...
// Section B {{{1
... code ...
// }}}1
```

4.2 Fold Commands (Normal Mode)

Once folds are detected, use `z`-prefixed commands in normal mode to manipulate them:

Command	Description
`zo`	Open the fold at the cursor
`zc`	Close the fold at the cursor
`za`	Toggle the fold at the cursor (open ↔ close)
`zR`	Open all folds in the buffer
`zM`	Close all folds in the buffer

These commands operate on the fold that contains the current cursor line. If the cursor is not inside a fold, a message is displayed.

4.3 Fold Gutter Indicators

When folds are active, the left gutter displays indicators showing fold state at a glance:

Command	Description
`+`	Start of a collapsed (closed) fold
`-`	Start of an open fold

`|`	Body of an open fold

A `+` in the gutter means that fold is collapsed — the lines between the fold start and end are hidden. Use `zo` or `za` on that line to expand it.

Screenshot: *Fold gutter showing +/-/| indicators next to collapsed and open folds in a Java file*

4.4 Fold Persistence

Fold state is automatically saved to `.foldstate` files in the same directory as the source file. When you reopen a file, Javi restores the previous fold state (which regions were collapsed or open).

The `.foldstate` file is a plain-text file with one fold per line in the format `startLine:endLine:collapsed`. These files can be safely deleted to reset fold state, and they are regenerated the next time you run a fold detection command.

4.5 Search and Folds

When you search (`/pattern` or `?pattern`), Javi automatically opens any collapsed folds that contain match results. This ensures search hits are always visible, even in heavily folded files.

4.6 Tips

- Start with `:fold` for brace-delimited files (JSON, Java, C) and `:foldindent` for indentation-structured files (Python, YAML).
- Use `zM` to collapse everything, then `zo` to selectively open regions of interest — this provides a quick overview of large files.
- Fold markers (`\{\{\{/\}\}\}`) are ideal for hand-curated sections in configuration files or documentation.
- Delete the `.foldstate` file to reset folds for a file.

5 Shell / Terminal

Javi includes a full integrated VT100 terminal emulator. You can run shell sessions, SSH connections, and interactive programs (vim, htop, etc.) without leaving the editor.

5.1 Starting a Shell

Command	Description
`F8`	Open or toggle to shell
`:vt`	Open or toggle to shell (same as F8)
`:shellnew`	Create a new shell session

The first time you press `F8`, a new shell session starts. The terminal runs under `script` for PTY support, with `TERM=xterm`, `COLUMNS`, and `LINES` sent automatically.

Screenshot: *Integrated terminal showing a shell session inside the Javi editor window*

5.2 Passthrough Mode

In a shell buffer, `F8` toggles **passthrough mode**:

- **Normal mode** — vi keybindings are active over the shell output. You can scroll, search, yank text, and use editing commands (delete, change, etc.) to modify the scrollback buffer.
- **Passthrough mode** — all keystrokes go directly to the shell. Use this for interactive programs (vim, top, etc.). Press `F8` again to return to normal mode.

You can also press `i` in a shell buffer to enter passthrough mode.

5.3 Managing Multiple Shells

Command	Description

`:shells`	List all active shells in a buffer
`:shellnext`	Switch to next shell
`:shellprev`	Switch to previous shell
`:shellname _name_`	Rename current shell
`:shellclose`	Close current shell and kill its process
`:shellclose _n_`	Close shell by ID
`:shellenv K=V`	Set environment variable in current shell
`:shellhistory`	Open full scrollback in read-only buffer

`ZZ` in a shell buffer closes the shell and kills the process.

The shell list appears in the plugin list (`F6`) once at least one shell has been created.

> **Tip:** Use `:shellname` to give meaningful names to your shells (e.g., "build", "test", "ssh-prod") for easier identification in the shell list.

6 AI Integration (Copilot)

Javi integrates with GitHub Copilot for AI-assisted coding: interactive chat, code explanation, code review, documentation generation, inline code completion with ghost text, and tool-augmented responses.

6.1 Setup

6.1.1 Loading the AI Plugin

The AI plugin is a separate module loaded via `.javini`:

```
loadclass javi.ai.AICommands
```

The AI source set is compiled as part of the standard Gradle build and included in the fat JAR (`javi-all.jar`).

6.1.2 Authentication

Authenticate with GitHub Copilot using the device flow:

```
:ai auth
```

This initiates the OAuth device flow — Javi displays a URL and a one-time code. Open the URL in a browser, enter the code, and authorize the application. The access token is stored in memory for the session.

6.1.3 Configuration

AI settings use the `ai.` prefix with the `:set` command:

Setting	Description
`ai.provider`	Provider: `copilot` (default), `openai`, or `anthropic`
`ai.model`	Model identifier (provider-specific default if unset)
`ai.apikey`	API key (or set `OPENAI_API_KEY` / `ANTHROPIC_API_KEY` env var)
`ai.maxTokens`	Maximum response token length (default: 2048)

Example:

```
:set ai.provider=copilot
:ai auth
:ai test
```

6.2 Commands

All AI commands use the `:ai` prefix:

Command	Description

`:ai _message_`	Send a chat message directly
`:ai chat`	Open interactive chat prompt
`:ai explain`	Explain code in the current buffer
`:ai review`	Review current buffer for bugs and issues
`:ai doc`	Generate Javadoc for current buffer code
`:ai complete`	Request inline code completion (ghost text)
`:ai accept`	Accept ghost text completion
`:ai dismiss`	Dismiss ghost text completion
`:ai cancel`	Cancel an in-flight AI request
`:ai refactor _instruction_`	Refactor code with instructions
`:ai config`	Display current AI configuration
`:ai clear`	Clear conversation history and chat buffer
`:ai test`	Test provider connectivity
`:ai auth`	Authenticate with Copilot (device flow)
`:ai models`	List available Copilot models
`:ai status`	Show request tracking and history
`:ai tools`	List registered AI tools
`:ai help`	Show AI command help in chat buffer

6.3 Normal Mode Key Bindings (ga prefix)

In normal mode, the `g` key followed by `a` enters an AI sub-mode. The third key selects the command:

Command	Description
`g a r`	Review current code
`g a e`	Explain current code
`g a d`	Generate documentation
`g a f`	Refactor (prompts for instruction)
`g a c`	Open AI chat
`g a s`	Show AI status
`g a t`	Test AI connection
`g a m`	List available models
`g a x`	Cancel AI request
`g a ?`	Show AI help

The `gg` sequence (go to first line) continues to work as expected.

6.4 Insert Mode Completion

While in insert mode, AI-powered code completion is available:

Command	Description
`Tab`	Trigger AI completion (if text before cursor)
`Tab` (ghost visible)	Accept ghost text completion
`Escape`	Dismiss ghost text / exit insert mode

When a completion arrives, it appears as "ghost text" — dimmed text after the cursor showing the suggested insertion. Press `Tab` to accept the ghost text, or `Escape` to dismiss it and return to command mode.

6.5 AI Tools

The AI system includes tool-use support, allowing the model to read files and buffers during conversations:

- **BufferInfoTool** — provides current buffer metadata
- **BufferReadTool** — reads lines from the current buffer
- **BufferWriteTool** — inserts or replaces text in the buffer
- **FileListTool** — lists files in the project
- **FileReadTool** — reads file contents from disk
- **GrepTool** — searches files by pattern

Tools are invoked automatically by the AI model during chat interactions when it needs additional context to answer your question.

6.6 Chat Buffer

AI responses appear in a dedicated `*ai-chat*` buffer. The buffer is created on first use and reused across interactions. Conversation history persists across `:ai chat` calls until `:ai clear` is invoked.

One-shot commands (`:ai explain`, `:ai review`, `:ai doc`) do not modify the conversation history.

Chat requests run asynchronously on a background thread — the editor remains responsive while waiting for a response.

6.7 Example Session

```
:ai auth                          " authenticate with Copilot
:ai test                          " verify connectivity
:ai What is a record in Java?     " ask a question
:ai explain                       " explain current file
:ai review                        " review current file for issues
:ai doc                           " generate javadoc
:ai status                        " see request history
:ai clear                         " reset conversation
```

> **Note:** The `:ai review` command may take longer for large files due to the volume of context sent to the server. The editor remains usable during the request.

7 Git Integration

Javi provides Magit-inspired Git integration through colon commands. All commands require Git to be installed and the editor to be running inside a Git repository.

A convenient shorthand is available: `:git _subcmd_` expands to `:git\_subcmd_`. For example, `:git status` is equivalent to `:git_status`, and `:git log` to `:git_log`.

7.1 Status and Staging

Command	Description
`:git_status`	Show staged, unstaged, and untracked files
`:git_stage _file_`	Stage a file (`git add`)
`:git_unstage _file_`	Unstage a file (`git restore --staged`)
`:git_stage_line`	Stage the file on the cursor line (in status buffer)
`:git_unstage_line`	Unstage the file on the cursor line
`:git_toggle`	Toggle staged/unstaged for cursor line
`:git_discard`	Discard unstaged changes for cursor line
`:git_refresh`	Refresh the status buffer

7.2 Committing

Command	Description
`:git_commit`	Open commit message editor showing staged changes
`:git_do_commit`	Finalize commit with message from the commit buffer

`:git_amend`	Amend the most recent commit

7.3 Viewing

Command	Description
`:git_diff`	Show `git diff` output in a buffer
`:git_diff _file_`	Show diff for a specific file
`:git_log`	Show last 30 log entries (oneline, graph)
`:git_branch`	Show all branches with latest commit
`:git_show`	Show commit details
`:git_blame`	Show `git blame` for the current file

7.4 Branch Operations

Command	Description
`:git_branch_create _name_`	Create a new branch
`:git_branch_switch _name_`	Switch to another branch
`:git_branch_delete _name_`	Delete a merged branch
`:git_merge _branch_`	Merge a branch into current
`:git_rebase _branch_`	Rebase current branch onto branch
`:git_rebase --continue`	Continue rebase after resolving conflicts
`:git_rebase --abort`	Abort an in-progress rebase

7.5 Remote Operations

Command	Description
`:git_fetch`	Fetch from remote
`:git_pull`	Pull from remote (fetch + merge)
`:git_push`	Push to remote

7.6 Stash

Command	Description
`:git_stash`	Stash working directory changes
`:git_stash_pop`	Pop the top stash entry
`:git_stash_list`	Show stash list in a buffer

7.7 Hunk-Level Operations

For fine-grained staging, the following commands work in diff and patch buffers:

Command	Description
`:git_stage_hunk`	Stage the hunk at the cursor
`:git_unstage_hunk`	Unstage the hunk at the cursor
`:git_patch`	Open patch view for the file at cursor
`:git_goto_file`	Jump to the source file from a diff or status line

7.8 Status Buffer

The `:git_status` command opens a `*git-status*` buffer showing the repository state:

```
Head: main (abc1234 "Last commit message")
Push: origin/main (up to date)
```

```
Staged changes (2)
  modified   src/main/java/javi/Example.java
  new file   src/main/java/javi/NewFile.java

Unstaged changes (1)
  modified   README.md

Untracked files (2)
  tmp/test.txt
  notes.md
```

Output buffers (`*git-status*`, `*git-diff*`, `*git-log*`, `*git-branch*`) are standard read-only editor buffers navigable with normal vi keys.

Screenshot: *Git status buffer showing staged, unstaged, and untracked files*

The status buffer auto-refreshes when files are saved in the editor.

7.9 Log Buffer

The `:git_log` buffer displays commit history with a graph. Within the log buffer, these commands provide interactive navigation:

Command	Description
`:git_log_diff`	Toggle inline diff for commit at cursor
`:git_expand`	Expand the commit at cursor (show diff)
`:git_expand_all`	Expand all commits
`:git_collapse_all`	Collapse all expanded commits

7.10 Workflow Example

1. Run `:git_status` to see the current repository state.
2. Stage files with `:git_stage src/main/java/javi/Example.java`.
3. Or use `:git_stage_line` on a file in the status buffer.
4. Run `:git_commit` to open the commit message editor.
5. Edit the commit message, then run `:git_do_commit` to finalize.
6. View the log with `:git_log` to verify your commit.
7. Push with `:git_push` when ready.

Note: If the current directory is not inside a Git repository, all git commands report "Not a git repository".

8 LSP Integration (Language Server Protocol)

Javi includes a Language Server Protocol client for modern IDE features: go-to-definition, find references, hover information, diagnostics, and code completion. LSP support works with any standard language server.

8.1 Setup

The LSP plugin is loaded via `.javini`:

```
loadclass javi.lsp.LspCommands
```

Once loaded, LSP commands and keybindings are immediately available. Javi starts the appropriate language server automatically when you open a file whose extension matches a configured server.

8.2 Commands

Command	Description
`:lspdef`	Go to definition of symbol under cursor
`:lspref`	Find all references to symbol under cursor

`:lsphover`	Show hover information (type, documentation)
`:lspdiag`	Show diagnostics (errors, warnings) for current file
`:lspcomp`	Trigger code completion at cursor
`:lspstatus`	Show LSP server status
`:lsprestart`	Restart the LSP server for the current file type
`:lsptoggle`	Enable/disable LSP for the current session
`:lspconfig`	Show or set language server configuration

8.3 Default Key Bindings

Command	Description
`F12`	Go to definition (`:lspdef`)
`Shift-F12`	Find references (`:lspref`)
`Ctrl-K`	Hover information (`:lsphover`)
`F9`	Code completion (`:lspcomp`)
`Ctrl-]`	Go to definition (LSP first, ctags fallback)

8.4 Integration with `:ta` and `Ctrl-]`

When the LSP plugin is loaded, it registers as a `TagLookupProvider`. This means `:ta` and `Ctrl-]` automatically route through LSP first:

1. If an LSP server is running for the current file type, the request goes to LSP (`textDocument/definition`).
2. If LSP finds a definition, navigation happens immediately.
3. If LSP has no result (or no server is running), the lookup falls through to the standard ctags system.

This integration is transparent — you use the same keys and commands you always have, and get LSP precision when available.

8.5 How LSP Works

When you open a file, Javi checks if a language server is configured for the file's extension. If found, it starts the server process and communicates via the standard LSP JSON-RPC protocol over stdio.

The LSP client handles:

- **Document synchronization** — edits are sent incrementally to the server as you type.
- **Request/response** — definition, references, hover, and completion requests use the LSP message protocol.
- **Diagnostics** — the server pushes error and warning information which Javi displays on request via `:lspdiag`.

8.6 Configuring Language Servers

Javi includes built-in configurations for common language servers. The server binary must be installed on your system.

8.6.1 Built-in Server Configurations

Language	Server	Command	Root Pattern
Java	Eclipse JDT-LS	`jdtls`	`build.gradle`
TypeScript/JS	typescript-language-server	`typescript-language-server --stdio`	`package.json`
Python	Pyright	`pyright-langserver --stdio`	`pyproject.toml`
C/C++	clangd	`clangd`	`compile_commands.json`

Javi searches common installation paths for each server (`/opt/homebrew/bin`, `/usr/local/bin`, `/usr/bin`).

8.6.2 Installing Language Servers

Java (Eclipse JDT Language Server):

```
# macOS (Homebrew)
brew install jdtls
```

C/C++ (clangd):

```
# macOS - usually included with Xcode Command Line Tools
# or install via Homebrew:
brew install llvm
# clangd is at /opt/homebrew/opt/llvm/bin/clangd
```

TypeScript/JavaScript:

```
npm install -g typescript-language-server typescript
```

Python (Pyright):

```
npm install -g pyright
# or
pip install pyright
```

8.6.3 Custom Server Configuration

Server configurations are persisted in `~/.javi/lsp.conf`. You can add or override servers in this file. The format is one server per line:

```
language_id = command | extensions | root_pattern
```

Example:

```
rust = rust-analyzer | .rs | Cargo.toml
go = gopls | .go | go.mod
```

Use `:lspstatus` to verify which servers are available and running.

> **Tip:** Javi can use the same language servers that VS Code uses. VS Code's language servers are standalone processes speaking the LSP protocol — they work identically with Javi.

8.6.4 Project Root Detection

When starting a language server, Javi walks up the directory tree from the current file looking for the *root pattern* file (e.g., `build.gradle`, `package.json`). This directory becomes the project root sent to the language server in the `initialize` request.

If no root pattern file is found, the directory containing the current file is used as the project root.

9 Appendix A: Comprehensive Key Binding Reference

This appendix provides a complete reference of every key binding and colon command registered in Javi, organized by category. Bindings are derived directly from the source code (`MapEvent.java`, `KeyMap.java`, and the various `Rgroup` command classes).

9.1 Modes

Command	Description
`i` `a` `o` `O`	Enter insert mode
`v` / `V`	Enter visual mode (char / line)
`R`	Enter replace (overwrite) mode
`Escape`	Return to command mode
`:`	Enter ex command line
`i` (in shell)	Enter shell passthrough mode

9.2 Normal Mode — Movement Keys

Movement keys can be used standalone or as the *motion* argument to operators like `d`, `c`, `y`, and `>`/`<`.

9.2.1 Cursor Movement

Key	Action

`h` / `Backspace`	Move cursor left
`l`	Move cursor right
`j` / `Down`	Move cursor down
`k` / `Up`	Move cursor up
`Left`	Move cursor left
`Right`	Move cursor right
`Space`	Move over (right, non-editing)

9.2.2 Word Movement

Key	Action
`w`	Forward to start of word
`W`	Forward to start of WORD (non-whitespace)
`b`	Backward to start of word
`B`	Backward to start of WORD
`e`	Forward to end of word
`E`	Forward to end of WORD
`Ctrl-Left`	Backward word
`Ctrl-Right`	Forward word

9.2.3 Line Position

Key	Action
`0`	Start of line (column 0)
`^`	First non-blank character
`$` / `End`	End of line
`|`	Go to column *n* (with count)
`+` / `Enter`	Move to first non-blank of next line
`-`	Move to first non-blank of previous line
`Home`	Start of line

9.2.4 Jumping and Go-To

Key	Action
`G`	Go to last line (or line *n* with *n*`G`)
`Shift-Home` / `Ctrl-Home`	Go to first line
`Ctrl-End`	Go to last line
`Shift-End`	Go to last line
`H`	Move to top of screen
`M`	Move to middle of screen
`L`	Move to bottom of screen
`%`	Jump to matching bracket

9.2.5 Scrolling

Key	Action
`Ctrl-F` / `PgDn`	Scroll forward one full page
`Ctrl-B` / `PgUp`	Scroll backward one full page
`Ctrl-D`	Scroll forward half page

`Ctrl-U`	Scroll backward half page
`Ctrl-E` / `Ctrl-Down`	Scroll one line down
`Ctrl-Y` / `Ctrl-Up`	Scroll one line up

9.2.6 Character Search on Line

Key	Action
`f` *char*	Find *char* forward on current line
`F` *char*	Find *char* backward on current line
`t` *char*	Move to just before *char* forward
`T` *char*	Move to just after *char* backward
`;`	Repeat last f/F/t/T
`,`	Repeat last f/F/t/T in opposite direction

9.2.7 Search

Key	Action
`/` *pattern*	Search forward for *pattern* (Java regex)
`?` *pattern*	Search backward for *pattern*
`n`	Repeat search in same direction
`N`	Repeat search in opposite direction
`Ctrl-F3`	Search forward (via edit key)

9.2.8 Regex-Based Motion (Sentences, Paragraphs, Sections)

Key	Action
`)`	Forward to next sentence (`.` or `.$`)
`(`	Backward to previous sentence
`}`	Forward to next blank line (paragraph)
`{`	Backward to previous blank line
`]`	Forward to next section (line starting with non-space + `{`)
`[`	Backward to previous section

9.2.9 Marks

Key	Action
`m` *char*	Set mark *char* at current position
`'` *char*	Jump to mark *char*

9.3 Normal Mode — Editing Keys

9.3.1 Entering Insert Mode

Key	Action
`i`	Insert before cursor
`I`	Insert at first non-blank of line
`a`	Append after cursor
`A`	Append at end of line
`o`	Open new line below
`O`	Open new line above
`s`	Substitute character (delete + insert)

`S`	Substitute entire line
`R`	Enter replace (overwrite) mode
`Insert`	Enter insert mode

9.3.2 Deleting

Key	Action
`x` / `Delete`	Delete character under cursor
`X` / `Backspace`	Delete character before cursor
`d` *motion*	Delete with motion (e.g., `dw`, `d$`)
`dd`	Delete entire line
`D` / `Shift-Delete`	Delete from cursor to end of line

9.3.3 Changing

Key	Action
`c` *motion*	Change with motion (delete + insert)
`cc`	Change entire line
`C`	Change from cursor to end of line
`r` *char*	Replace single character under cursor
`~`	Toggle case of character under cursor

9.3.4 Yank and Put

Key	Action
`y` *motion*	Yank (copy) with motion
`yy` / `Y`	Yank entire line
`p`	Put (paste) after cursor
`P`	Put (paste) before cursor
`"` *reg*	Use named register *reg* for next yank/put

9.3.5 Undo and Redo

Key	Action
`u`	Undo last change
`Ctrl-Z`	Undo (alternate)
`Alt-Backspace`	Undo (alternate)
`U`	Undo all changes on current line
`Ctrl-R`	Redo
`Ctrl-Y`	Redo (alternate)
`Ctrl-Shift-Z`	Redo (alternate)
`Alt-Shift-Backspace`	Redo (alternate)

9.3.6 Visual Mode

Key	Action
`v`	Enter character-wise visual mode
`V`	Enter line-wise visual mode
`Escape`	Exit visual mode
`d`	Delete selection
`y`	Yank selection

`c`	Change selection
`>` / `<`	Shift selection right / left
`~`	Toggle case of selection

9.3.7 Shift / Indent

Key	Action
`>>`	Shift current line right
`<<`	Shift current line left
`Shift-Up`	Shift-move up (for line shifting)
`Shift-Down`	Shift-move down (for line shifting)

9.3.8 Miscellaneous Normal Mode

Key	Action
`.`	Repeat last editing command
`J`	Join current line with next
`z`	Z-position scroll (`zt`, `zz`, `zb`) and fold commands (`zo`, `zc`, `za`, `zR`, `zM`)
`Z`	Z-process (ZZ = save and quit, ZQ = quit)
`Ctrl-L`	Redraw screen
`Ctrl-G`	Show file status information
`:`	Enter ex command line
`Alt-J`	Evaluate current file as JavaScript

9.3.9 Folding

Key	Action
`zo`	Open fold at cursor
`zc`	Close fold at cursor
`za`	Toggle fold at cursor
`zR`	Open all folds
`zM`	Close all folds

9.4 Function Keys

Function keys provide quick access to Javi features. Some have modifier-key variants.

Key	Modifier	Action
`F1`	—	Next position in position list
`F1`	Shift	Toggle context-sensitive help panel
`F1`	Ctrl	Next position with wait
`F2`	—	File list (open buffers)
`F3`	—	Directory list
`F3`	Ctrl	Search forward (regex search)
`F4`	—	Font list
`F5`	—	Position list
`F6`	—	Plugin list
`F7`	—	Make (build)
`F8`	—	Open / toggle shell
`F10`	—	Communication channel

`F11`	—	Toggle fullscreen

9.5 Tags and Navigation

Key	Action
`Ctrl-]`	Jump to definition (LSP first, then ctags)
`Ctrl-T`	Pop tag stack (return to previous location)
`Ctrl-^`	Switch to next file in file list
`F12`	Go to definition (LSP)
`Shift-F12`	Find references (LSP)
`Ctrl-K`	Hover information (LSP)
`F9`	Code completion (LSP)

9.6 Insert Mode Keys

While in insert mode, these special keys are active:

Key	Action
`Escape` / `Ctrl-]`	Complete and exit insert mode (dismisses ghost text)
`Tab`	Trigger AI completion or accept ghost text (falls back to insert tab)
`Backspace`	Delete character before cursor
`Delete`	Delete character under cursor
`Insert`	Toggle insert/overwrite mode
`Up`	Move to previous line (while inserting)
`Down`	Move to next line (while inserting)
`Ctrl-V`	Verbatim (insert next character literally)
`Ctrl-P`	Put (paste) buffer contents

9.7 Buffer-Specific Overlay Keymaps

Different buffer types activate overlay keymaps that modify or extend the normal-mode bindings.

9.7.1 File List (F2)

When viewing the file list buffer, these bindings replace their normal-mode equivalents:

Key	Action
`Enter`	Open file at cursor (replaces move-to-next-line)
`F1`	Open file at cursor
`Ctrl-F1`	Open and show position list
`Shift-F1`	Open in split view
`F2`	Return to previous buffer

All other normal-mode keys remain active.

9.7.2 Directory Editor (DirEdit)

When viewing a directory, these bindings are active:

Key	Action
`Enter`	Open file or enter directory
`-`	Go to parent directory
`.`	Toggle hidden (dot) files
`s`	Cycle sort mode (name / size / date / type)

`S`	Toggle directory in/out of search path
`R`	Refresh listing
`D`	Delete file under cursor
`o` / `O`	Create new file or directory (inline prompt)
`q`	Quit directory browser

9.7.3 Shell Buffer

When viewing a shell (VT100) buffer in normal mode:

Key	Action
`i` / `Insert`	Enter passthrough mode (keys go to shell)
`F8`	Toggle passthrough mode
`ZZ`	Close shell and kill process

9.8 Ex Commands Reference

All colon commands available in Javi. Type `:` followed by the command name and press Enter.

9.8.1 File Operations

Command	Description
`:e _file_`	Edit (open) file
`:e!`	Reload current file, discarding changes
`:vi _file_`	Edit file (alias for `:e`)
`:w`	Write (save) file
`:w _file_`	Write to named file
`:wq`	Write and quit
`:x`	Write (if changed) and quit
`:q`	Quit (fails if unsaved changes)
`:q!`	Quit without saving
`:r _file_`	Read file contents into buffer
`:r <_cmd_`	Read shell command output into buffer

9.8.2 Line Range Commands (Ex Mode)

These commands accept line number ranges (e.g., `:3,7d`):

Command	Description
`:_n_`	Go to line *n*
`:_n_,_m_ d`	Delete lines *n* through *m*
`:_n_,_m_ y`	Yank lines *n* through *m*
`:_n_,_m_ m _k_`	Move lines to after line *k*
`:_n_,_m_ t _k_` / `:_n_,_m_ co _k_`	Copy lines to after line *k*
`:s/old/new/`	Substitute first occurrence on current line
`:s/old/new/g`	Substitute all on current line
`:%s/old/new/g`	Substitute all in entire file
`:g/_pattern_/d`	Delete all lines matching *pattern*
`:v/_pattern_/d`	Delete all lines NOT matching *pattern*

9.8.3 Editor Settings

Command	Description

`:set _option_=_value_`	Set an editor option
`:tabstop _n_`	Set tab display width
`:lines _n_`	Set default window height
`:setwidth _n_`	Set default window width

9.8.4 Shell / Terminal Commands

Command	Description
`:vt`	Open or toggle to shell (same as F8)
`:shellnew`	Create a new shell session
`:shells`	List all active shells
`:shellnext`	Switch to next shell
`:shellprev`	Switch to previous shell
`:shellname _name_`	Rename current shell
`:shellclose`	Close current shell
`:shellclose _n_`	Close shell by ID
`:shellenv K=V`	Set environment variable in current shell
`:shellhistory`	Open full scrollback in read-only buffer

9.8.5 Tags and Navigation

Command	Description
`:ta _tag_`	Jump to tag (LSP first, then ctags)
`:gototag`	Jump to tag under cursor
`:poptag`	Pop tag stack
`:cn`	Go to next position in position list
`:cp`	Go to previous position in position list

Not yet implemented: `:tagsauto`, `:tagfiles`, and `:tagadd` are planned but not yet available.

9.8.6 LSP Commands

Command	Description
`:lspdef`	Go to definition
`:lspref`	Find all references
`:lsphover`	Show hover information
`:lspcomp`	Trigger code completion
`:lspdiag`	Show diagnostics
`:lspstatus`	Show server status
`:lsprestart`	Restart LSP server
`:lsptoggle`	Enable/disable LSP
`:lspconfig`	Show/set server configuration

9.8.7 AI Commands

Command	Description
`:ai _message_`	Send chat message
`:ai chat`	Interactive chat prompt
`:ai explain`	Explain current code

`:ai review`	Review code for issues
`:ai doc`	Generate documentation
`:ai complete`	Request inline completion
`:ai accept`	Accept ghost text
`:ai dismiss`	Dismiss ghost text
`:ai cancel`	Cancel in-flight request
`:ai refactor _instruction_`	Refactor with instruction
`:ai config`	Show configuration
`:ai clear`	Clear chat history
`:ai test`	Test connection
`:ai auth`	Copilot device flow auth
`:ai models`	List available models
`:ai status`	Show request tracking
`:ai tools`	List registered tools
`:ai help`	Show AI help

9.8.8 Directory Editor Commands

Command	Description
`:diredit _path_`	Open directory editor
`:diredit_open`	Open file/directory under cursor
`:diredit_parent`	Go to parent directory
`:diredit_rename _name_`	Rename file under cursor
`:diredit_mkdir _name_`	Create new subdirectory
`:diredit_newfile _name_`	Create new empty file
`:diredit_copy _dest_`	Copy file under cursor
`:diredit_delete`	Delete file under cursor
`:diredit_mark`	Toggle delete mark on file
`:diredit_execute`	Execute marked operations
`:diredit_shell`	Open shell in current directory
`:diredit_create`	Create new file or directory (inline prompt)

9.8.9 Key Binding Customization

Command	Description
`:mapkey _group_ _key_ _command_`	Bind a key in group
`:unmapkey _group_ _key_`	Unbind a key
`:map`	Show all key bindings for current context
`:map _keymap_`	Show bindings for named keymap
`:keymap`	Show active keymap chain
`:savemapkeys`	Save user bindings to `~/.javi/keybindings`
`:loadmapkeys`	Load user bindings from disk

For `:mapkey`, *group* is `move` or `edit`. *key* can be a single character, `C-x` (Ctrl), `S-x` (Shift), or names like `F1`–`F12`, `Up`, `Down`, `Home`, `End`, `PgUp`, `PgDn`, `Insert`, `Delete`.

9.8.10 Build and Compile

Command	Description

`:mk`	Run make (build via make.pl)
`:cc`	Run cc (compile via make.pl)
`:comp`	Compile with Java compiler
`:compa`	Compile all with Java compiler
`:cstyle`	Run Checkstyle on current file

9.8.11 Folding Commands

Command	Description
`:fold`	Detect folds by brace/bracket matching (syntax-based)
`:foldindent`	Detect folds by indentation level
`:foldindent _n_`	Detect folds with *n*-space indent levels
`:foldmarker`	Detect folds by `\{\{\{/\}\}\}` markers

See Section 4 for details on fold detection methods and normal-mode fold commands.

9.8.12 Help System

Command	Description
`:help`	Show help index (context-sensitive)
`:help _topic_`	Show help for specific topic
`:contexthelp`	Toggle context-sensitive help side panel
`:helpscrolldown`	Scroll help panel down
`:helpscrollup`	Scroll help panel up

Available topics: `index`, `movement`, `editing`, `search`, `files`, `ex`, `visual`, `undo`, `window`, `shell`, `diredit`, `filelist`, `directory`, `keybindings`, `folding`, `git`.

9.8.13 Git Integration Commands

Command	Description
`:git _subcmd_`	Shorthand: expands to `:git\ _subcmd_`
`:git_status`	Show staged, unstaged, and untracked files
`:git_stage _file_`	Stage a file
`:git_unstage _file_`	Unstage a file
`:git_stage_line`	Stage file on cursor line
`:git_unstage_line`	Unstage file on cursor line
`:git_toggle`	Toggle staged/unstaged for cursor line
`:git_discard`	Discard unstaged changes for cursor line
`:git_refresh`	Refresh status buffer
`:git_commit`	Open commit message editor
`:git_do_commit`	Finalize commit
`:git_amend`	Amend most recent commit
`:git_diff`	Show diff
`:git_log`	Show log entries
`:git_branch`	Show branches
`:git_show`	Show commit details
`:git_blame`	Show blame for current file
`:git_branch_create`	Create branch
`:git_branch_switch`	Switch branch

`:git_branch_delete`	Delete branch
`:git_merge`	Merge branch
`:git_rebase`	Rebase current branch
`:git_fetch`	Fetch from remote
`:git_pull`	Pull from remote
`:git_push`	Push to remote
`:git_stash`	Stash changes
`:git_stash_pop`	Pop stash
`:git_stash_list`	List stashes
`:git_stage_hunk`	Stage hunk at cursor
`:git_unstage_hunk`	Unstage hunk at cursor
`:git_patch`	Open patch view
`:git_goto_file`	Jump to file from diff/status

9.8.14 JavaScript Integration

Command	Description
`:jseval _expr_`	Evaluate JavaScript expression
`:jsevalfile`	Evaluate current file as JavaScript
`:jsclear`	Clear JavaScript output buffer

9.8.15 Miscellaneous

Command	Description
`:!`	Run external shell command
`:fl`	Go to file list
`:rep`	Report / repeat
`:te`	Toggle editor state
`:loadgroup`	Load command group
`:loadclass _classname_`	Load a plugin class by name
`:persistfile _name_`	Set persistent file name
`:tabfix`	Fix tab/space indentation
`:fullscreen`	Toggle fullscreen mode

10 Appendix B: `.javini` Configuration Reference

The `.javini` file is Javi's startup configuration file. It is read from the current directory when Javi launches. Each line in the file is executed as if it were typed at the colon command prompt, so any valid colon command can appear in `.javini`.

10.1 File Location

Javi looks for `.javini` in the working directory where it was launched. There is no global (home directory) configuration file — each project can have its own `.javini` with project-specific settings.

10.2 Format

One command per line. Blank lines and lines that cause errors are silently skipped. Comments are not supported — every non-empty line is interpreted as a command.

```
fontname Verdana
fontsize 15.5
fontweight 1.0
lines 70
tabstop 3
```

```
loadplugin javi-hello.jar
loadclass javi.git.GitCommands
loadmapkeys
```

10.3 Common Settings

Setting	Description
`fontname _name_`	Set the editor font face (e.g., `Verdana`, `Courier New`, `JetBrains Mono`)
`fontsize _n_`	Set font size in points (float, e.g., `15.5`)
`fontweight _n_`	Set font weight (float: `1.0` = normal, `2.0` = bold)
`monofontname _name_`	Set the monospace font for terminals
`lines _n_`	Set default window height in lines
`setwidth _n_`	Set default window width in columns
`tabstop _n_`	Set tab display width (default: 8)

10.4 Plugin Loading

Setting	Description
`loadplugin _file.jar_`	Load a plugin JAR file from the current directory or classpath
`loadclass _classname_`	Load a command class by fully qualified name (e.g., `javi.git.GitCommands`)
`loadgroup _name_`	Load a named command group

Plugins are loaded during startup after the main editor initializes. A plugin JAR must contain classes that register commands via the Javi command framework.

10.5 Key Binding Customization

Setting	Description
`mapkey _group_ _key_ _command_`	Bind a key in the named group
`unmapkey _group_ _key_`	Remove a key binding
`loadmapkeys`	Load saved key bindings from `~/.javi/keybindings`

Add `loadmapkeys` to your `.javini` to restore custom bindings automatically on startup.

10.6 Editor Options via `set`

The `set` command can also appear in `.javini` for any registered option:

```
set ai.provider=copilot
set ai.model=gpt-4
```

See the AI Integration section for available `ai.*` options.

10.7 Example `.javini`

A typical development configuration:

```
fontname JetBrains Mono
fontsize 14.0
fontweight 1.0
lines 60
setwidth 120
tabstop 4
loadclass javi.lsp.LspCommands
loadclass javi.ai.AICommands
loadclass javi.git.GitCommands
loadmapkeys
```

Tip: Keep your `.javini` minimal. Use it for font/size preferences and essential plugin loading. Per-session settings are better managed interactively with colon commands.

11 Appendix C: Accuracy Notes

The following features described in this manual are planned but **not yet implemented** in the codebase as of May 2026. The corresponding sections describe the intended design:

- **Tag management commands** — The commands `:tagsauto` (auto-regenerate ctags), `:tagfiles` (list tag files), `:tagadd` (add tag file), and `:gid` (find references via mkid) are planned but not yet registered as commands. Basic tag lookup (`:ta`, `Ctrl-]`, `Ctrl-T`) and automatic mkid cross-referencing work.
- **`:shell` alias** — The `:shell` command documented in help text is not registered as a colon command. Use `:vt` or press F8 to access the terminal. All other shell management commands (`:shellnew`, `:shells`, `:shellnext`, etc.) work as documented.

The following features are **fully implemented** and documented accurately:

- **AI Integration** (Section 6) — All `:ai` commands are implemented on `feature/F8-ai-integration`: chat, explain, review, doc, complete, accept, dismiss, cancel, refactor, auth, models, status, tools, help. Normal-mode `ga` prefix bindings and insert-mode Tab completion with ghost text are functional. Multi-provider support (Copilot, OpenAI, Anthropic) with tool-use (BufferRead, FileRead, Grep, etc.). Loaded via `loadclass javi.ai.AICommands` or the AI plugin JAR.
- **LSP Integration** (Section 8) — All LSP commands are implemented on `feature/F7-lsp-integration`: lspdef, lspref, lsphover, lspcomp, lspdiag, lspstatus, lsprestart, lsptoggle, lspconfig. Key bindings (F12, Shift-F12, Ctrl-K, F9) are registered by the plugin. TagLookupProvider integration routes `:ta` and `Ctrl-]` through LSP before ctags. Loaded via `loadclass javi.lsp.LspCommands`.
- **Context-Sensitive Help** (Section 1) — The `Shift-F1` help side panel, `:contexthelp`, and help panel scrolling commands are implemented. The panel dynamically reflects the active keymap.
- **Folding** (Section 4) — All fold detection methods (`:fold`, `:foldindent`, `:foldmarker`) and fold manipulation keys (`zo`/`zc`/`za`/`zR`/`zM`) are implemented on master.
- **Git Integration** (Section 7) — All git commands documented in this manual are implemented, including advanced commands: `git_stage_hunk`, `git_unstage_hunk`, `git_patch`, `git_blame`, `git_goto_file`, `git_amend`, `git_log_diff`, `git_expand`, `git_expand_all`, and `git_collapse_all`.

All other sections accurately reflect the current codebase.